RUNGTA COLLEGE OF ENGINEERING AND TECHNOLOGY, BHILAI

MINOR PROJECT – 1 | AIML TRACK

PROJECT REPORT

MediQueue AI

AI-Based Healthcare Appointment & Queue Prediction System

Submitted by

Mahi Kumari

B.Tech – Computer Science Engineering (Core)

Academic Session: 2026

GitHub Repository

https://github.com/Mahi200611/MediQueue-AI

Live Demonstration

https://mediqueue-ai-uhumtut69h2yrp4iceknmx.streamlit.app/

Edit with WPS Office

DECLARATION

I hereby declare that the project entitled “MediQueue AI – AI-Based Healthcare Appointment & Queue Prediction System” has been prepared for Minor Project 1 under the AIML track. It describes an academic prototype that applies machine-learning methods to estimate healthcare appointment queue position and waiting time.

The project is developed for educational and demonstration purposes. Predictions depend on the dataset and training process and must not be treated as guaranteed waiting times or as a substitute for hospital staff or a live queue-management system.

Student Name: Mahi Kumari

Signature: _____

Date: _____

Edit with WPS Office

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty members of Rungta College of Engineering and Technology, Bhilai, for the opportunity to undertake this Minor Project 1 under the AIML track. This project helped me explore the application of machine learning through data processing, model development, and web application deployment.

I also acknowledge the open-source communities behind Python, Pandas, NumPy, Scikit-learn, Joblib, Matplotlib, and Streamlit, whose tools support the development of this project.

ABSTRACT

MediQueue AI is an AI-based healthcare appointment and queue prediction prototype intended to estimate a patient's queue position and waiting time from appointment-related information. Uncertain waiting periods can inconvenience patients and make hospital visits difficult to plan. This project demonstrates how machine-learning methods can generate estimates from structured appointment data and present them through a web-based dashboard.

The application is built with Python and Streamlit. Pandas and NumPy support data handling, Scikit-learn provides machine-learning functionality, Joblib loads trained model files, and Matplotlib supports visual analytics. The repository includes application code, a training script, a CSV dataset, model files, and a dependency list. The interface includes Overview, AI Prediction, Analytics, and System Status sections.

The project is a proof of concept using a prepared demonstration dataset, not verified live hospital data. Predictions require evaluation and validation using representative real-world data before any operational healthcare use.

TABLE OF CONTENTS

- 1. Introduction
- 2. Problem Statement
- 3. Aim and Objectives
- 4. Scope of the Project
- 5. Existing and Proposed System
- 6. Technology Stack
- 7. Dataset Description
- 8. System Architecture
- 9. Methodology and Implementation
- 10. Application Modules
- 11. Testing and Results
- 12. Advantages and Limitations
- 13. Future Scope

Edit with WPS Office

- 14. Conclusion
- 15. References

Edit with WPS Office

1. INTRODUCTION

Hospitals and clinics manage scheduled appointments and walk-in patients throughout the day. Waiting time can vary because of appointment volume, consultation duration, emergency cases, and staff availability. Patients may therefore find it difficult to estimate their position in a queue or determine how long they must wait.

MediQueue AI is a web-based academic prototype designed to estimate queue position and waiting time using appointment-related inputs. Users enter information through an interactive dashboard, and the application uses saved machine-learning models to generate estimates. The project combines data handling, machine learning, visualization, and deployment in one demonstrable application.

2. PROBLEM STATEMENT

Patients often lack clear information about their likely queue position and expected waiting time. Manual estimates may be inconsistent and may not reflect patterns in previous appointments. This project explores whether trained machine-learning models can provide estimates from structured appointment data and display the results in an accessible web interface.

3. AIM AND OBJECTIVES

Aim: To develop a machine-learning-based prototype that estimates healthcare appointment queue position and waiting time through a web application.

- Prepare structured appointment data for model development.
- Use machine-learning models to estimate queue position and waiting time.
- Create an interactive interface for entering appointment details.
- Display prediction results clearly.
- Visualize selected patterns in the available dataset.
- Deploy the application online for demonstration and evaluation.

4. SCOPE OF THE PROJECT

The project is an academic prototype based on a CSV dataset and trained model files. It demonstrates data preparation, supervised learning, prediction, visualization, and web deployment. It is not connected to a live hospital appointment database, doctor schedule, or real-time queue. Outputs are estimates and must not be treated as confirmed appointment numbers.

5. EXISTING AND PROPOSED SYSTEM

5.1 Existing System

Patients may rely on scheduled appointment times or ask reception staff for waiting-time

Edit with WPS Office

estimates. Such estimates can become inaccurate when appointments run late, consultation times vary, or urgent cases affect the order.

5.2 Proposed System

MediQueue AI accepts appointment-related inputs through a Streamlit interface. The application prepares the inputs in the format expected by the saved models and displays estimated queue position and waiting time. An analytics section presents selected summaries of the demonstration dataset.

6. TECHNOLOGY STACK

- Python – main programming language.
- Pandas – loading and manipulating tabular data.
- NumPy – numerical operations.
- Scikit-learn – machine-learning functionality.
- Random Forest – ensemble approach used in the prediction workflow.
- Joblib – loading saved model objects.
- Streamlit – interactive web interface.
- Matplotlib – data visualization.
- CSV – dataset file format.
- Git and GitHub – version control and source-code hosting.
- Streamlit Community Cloud – online demonstration hosting.

7. DATASET DESCRIPTION

The project repository contains a dataset file named healthcare_appointments.csv. It is a prepared demonstration dataset and should not be described as real hospital data unless its origin and permissions have been independently verified.

The exact record count, feature names, data types, and target columns should be confirmed from the final CSV and training script. Depending on the implementation, appointment-related fields may include appointment time, department, patient category, previous visits, existing appointments, or emergency status. The intended outputs are estimated queue position and waiting time.

Data preparation involves loading the CSV, reviewing its structure and missing values, encoding categorical variables where necessary, and keeping the feature order consistent between training and prediction.

8. SYSTEM ARCHITECTURE

1. User Input: The user enters appointment-related details on the web interface.

Edit with WPS Office

2. Input Processing: The application arranges values into the format expected by the trained models.
3. Model Loading: Saved model files are loaded using Joblib.
4. Prediction: The models estimate queue position and waiting time.
5. Result Display: Estimates are shown in the application.
6. Analytics: Charts summarize selected patterns in the dataset.

System flow: User → Streamlit Interface → Input Processing → Trained ML Models → Queue/Waiting-Time Estimates → Results Dashboard.

9. METHODOLOGY AND IMPLEMENTATION

9.1 Data Loading and Exploration

The dataset is loaded using Pandas. Initial exploration checks record count, column names, data types, missing values, and distributions to identify data-quality issues.

9.2 Data Preprocessing

Categorical information may need to be encoded into numeric representations. The preprocessing used during training must also be applied consistently to new inputs. Missing-value handling and feature selection should follow the actual training script.

9.3 Model Training

The project uses a Random Forest-based approach. Random Forest combines multiple decision trees and can represent non-linear relationships between input features and outputs. The repository includes saved model files for queue-position and waiting-time prediction. The training script should be used to verify the exact targets and configuration.

9.4 Model Evaluation

A model should be evaluated on data not used to fit it. For numerical waiting-time predictions, common metrics include Mean Absolute Error (MAE) and R-squared (R^2). MAE measures average absolute prediction error, while R^2 compares the model with a baseline. Actual scores must be taken from the final training run; this report does not invent performance figures.

9.5 Application Development and Deployment

Streamlit provides the browser-based interface. The application loads model files, collects user inputs, runs predictions, and presents results and analytics. The source code is hosted on GitHub and the demonstration is hosted through Streamlit Community Cloud.

10. APPLICATION MODULES

10.1 Overview

Introduces MediQueue AI and summarizes the purpose and functions of the application.

Edit with WPS Office

10.2 AI Prediction

Collects appointment-related information and displays estimated queue position and waiting time returned by the models.

10.3 Analytics

Displays charts and summaries for exploring the dataset. These visuals do not by themselves establish real-world validity.

10.4 System Status

Provides a place to communicate the status of application components and resources. Indicators should reflect checks implemented in the application.

11. TESTING AND RESULTS

Functional testing should confirm that the dashboard opens, navigation works, input controls accept valid values, models load, prediction outputs appear, and charts render. The hosted application allows the project to be demonstrated in a browser.

- Interface test – verify each navigation section opens correctly.
- Input test – check required fields and valid input ranges.
- Model test – confirm saved models load and return outputs for valid inputs.
- Analytics test – confirm charts render from the dataset.
- Deployment test – open the live application and check the main workflow.

Insert screenshots from the final running application in this section. If evaluation metrics are available, add the actual values and explain the test procedure. No unverified model-performance result is claimed here.

12. ADVANTAGES AND LIMITATIONS

12.1 Advantages

- Provides estimates of queue position and waiting time.
- Demonstrates an end-to-end AI/ML prototype workflow.
- Separates application code and trained model files.
- Includes analytics for exploring demonstration data.
- Can be accessed through a web browser for demonstrations.

12.2 Limitations

- The dataset is not verified as representative of real hospital operations.
- Prediction quality depends on the training data.
- Actual waiting times may change due to emergencies, staffing, consultation duration, and

Edit with WPS Office

other factors.

- The prototype does not integrate with a live hospital queue.
- Predictions are not guaranteed and must not be used for medical decisions.

13. FUTURE SCOPE

- Integrate with a secure hospital appointment system and real-time queue data.
- Include doctor availability, consultation duration, holidays, and emergency handling.
- Validate models using representative, consented, de-identified data.
- Provide prediction ranges or uncertainty estimates.
- Add notifications, appointment booking, and administrative features.
- Introduce privacy controls, access permissions, audit logs, and monitoring before real-world use.

14. CONCLUSION

MediQueue AI demonstrates how machine-learning methods can be combined with a web interface to estimate healthcare appointment queue position and waiting time. It brings together structured data, preprocessing, Random Forest-based prediction, analytics, and online deployment. The prototype is suitable for academic demonstration of an AI/ML workflow. Before real-world use, it requires verified data, formal model evaluation, systematic testing, privacy safeguards, and integration with reliable live queue information.

15. REFERENCES

- Python Documentation: https://docs.python.org/3/
- Pandas Documentation: https://pandas.pydata.org/docs/
- NumPy Documentation: https://numpy.org/doc/
- Scikit-learn Documentation: https://scikit-learn.org/stable/
- Streamlit Documentation: https://docs.streamlit.io/
- Joblib Documentation: https://joblib.readthedocs.io/
- Matplotlib Documentation: https://matplotlib.org/stable/
- MediQueue AI GitHub repository: https://github.com/Mahi200611/MediQueue-AI
- MediQueue AI live demonstration: https://mediqueue-ai-uhumtut69h2yrp4iceknmx.streamlit.app/

Edit with WPS Office